

✓ 제조 AI 데이터 품질 관리 · 전처리 실습

이 실습에서는 제조 데이터를 읽고, **분석에 사용하기 전에 어떤 값을 확인하고 어떻게 정리하는지** 살펴봅니다. 데이터 확인 → 완전성 → 유효성 → 일관성 → 유일성 → 정확성 순서로 진행합니다. 각 단계에서 문제 데이터를 찾아 처리하며, 품질 점수를 별도로 계산하지는 않습니다.

실행 준비: `Chemical_Numeric_Data_Quality.csv`를 현재 작업 폴더에 준비하고 위에서부터 셀을 차례대로 실행하세요. Colab에서는 파일 영역에 CSV를 업로드한 뒤 실행하면 됩니다. 중간 단계부터 다시 시작하면 이미 수정된 `df`를 사용하므로, 처음 상태가 필요하다면 데이터 로딩부터 다시 실행하세요.

원본의 실행 코드와 저장된 출력은 유지하고 설명을 덧붙였습니다. 저장된 출력은 원본 실행 기록이며, 이번 편집에서 재실행한 결과는 아닙니다. 무작위 표본과 자료형 표시, 경고 등은 실행 환경에 따라 달라질 수 있습니다.

데이터 로딩 + 확인

먼저 **표의 구조, 열의 의미, 값의 범위**를 확인합니다.

이름	이 실습에서의 의미
<code>df</code>	행과 열을 가진 전체 표, DataFrame
<code>df.Temp</code> 또는 <code>df['Temp']</code>	온도 한 열, Series
<code>df.index</code>	행을 구분하는 인덱스 라벨. Lot 열과는 별개
<code>Lot</code>	로트 식별자
<code>Temp</code>	온도(°C)
<code>Viscosity</code>	점도(cP)
<code>Failure</code>	정상/불량 판정
<code>FailureCode</code>	불량 유형을 나타내는 코드

```
# pandas: 행과 열로 이루어진 표 데이터를 다루는 라이브러리
# pd라는 짧은 이름으로 불러온다.
import pandas as pd

# CSV 파일을 읽어 DataFrame(표)으로 만들고 df에 저장한다.
# encoding='cp949': 이 파일에 저장된 한글을 읽기 위한 문자 인코딩 지정
# 실행 전에 CSV 파일을 현재 작업 폴더에 준비한다.
df = pd.read_csv('Chemical_Numeric_Data_Quality.csv', encoding='cp'
```

```
# 실수 값을 화면에 표시할 때 소수점 넷째 자리까지 보이도록 설정한다.
# 저장된 수치나 계산 결과를 바꾸는 것이 아니라, 출력 모양만 바꾼다.
# lambda x: ... 는 입력 x를 받아 뒤의 식을 계산하는 짧은 함수이다.
# '%.4f' % x: x를 소수점 4자리의 문자열로 표현 (예: 2.3 → '2.3000')
pd.set_option('display.float_format', lambda x: '%.4f' % x)
```

```
# 전체 데이터에서 무작위로 3개 행을 뽑아 표(DataFrame)로 반환한다.
# 파일에 어떤 값이 들어 있는지 살펴본다. 실행할 때마다 뽑힌 행은 달라질 수 있
# 확인용 출력이므로 원래 df의 행이 줄어들지는 않는다.
df.sample(3)
```

	Lot	Temp(°C)	Viscosity(cP)	Failure	Failure code
457	2109326382	25.7100	2.3800	0	NaN
971	2110404864	25.0200	NaN	0	NaN
639	2110395093	24.9900	1.7200	0	NaN

```
# info(): 행 수, 열 이름, 결측치가 아닌 값의 개수, 자료형을 화면에 출력한다
# Non-Null Count가 전체 행 수보다 작으면 그 열에 결측치가 있다는 뜻이다.
# info() 자체의 반환값은 None이며, 요약 표를 반환하는 함수는 아니다.
df.info()
```

```
# describe(): 기본적으로 수치형 열의 요약 통계를 DataFrame으로 반환한다.
# count=유효 값 개수, mean=평균, std=표준편차, min/max=최솟값/최댓값
# 25%, 50%, 75%=사분위수 (50%는 중앙값)
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Lot                    1000 non-null  int64
1   Temp(°C)               870 non-null   float64
2   Viscosity(cP)          911 non-null   float64
3   Failure                 1000 non-null  object
4   Failure code           161 non-null   float64
dtypes: float64(3), int64(1), object(1)
memory usage: 39.2+ KB
```

	Lot	Temp(°C)	Viscosity(cP)	Failure code
count	1000.0000	870.0000	911.0000	161.0000
mean	2110175932.5680	31.4714	5.5346	3.1056
std	677478.9190	33.0484	11.8892	1.3991
min	2109021818.0000	2.4500	0.7400	1.0000
25%	2109585209.0000	24.3525	1.5700	2.0000
50%	2110189868.0000	25.4300	2.3900	3.0000
75%	2110782208.0000	26.4400	3.2500	4.0000
max	2111289797.0000	435.3500	101.8000	5.0000

```
# 기존 열의 순서에 맞춰 열 이름 5개를 한꺼번에 바꾼다.
# 데이터 값과 열 순서는 그대로이며, 이름만 짧게 정리한다.
# Lot=로트 식별자, Temp=온도, Viscosity=점도
# Failure=정상/불량 여부, FailureCode=불량 코드
df.columns = ['Lot', 'Temp', 'Viscosity', 'Failure', 'FailureCode']
```

```
# head(): 맨 앞의 5개 행을 DataFrame으로 반환한다. 열 이름이 바뀌었는지
# sample()은 무작위 행, head()는 앞쪽 행을 보여준다.
df.head()
```

	Lot	Temp	Viscosity	Failure	FailureCode
0	2110262249	24.4400	101.8000	1	2.0000
1	2109975633	23.9500	82.6300	1	NaN
2	2109024936	23.2400	1.5200	1	4.0000
3	2110009104	NaN	81.4900	1	NaN
4	2109169908	23.3300	71.9200	1	NaN

✓ 불량 코드의 빈칸과 자료형 정리

불량 코드는 크기를 계산하는 수치보다 **유형을 구분하는 이름표**에 가깝습니다. 이 실습에서는 문자열로 통일하고, 빈칸은 우선 'None'으로 표시합니다.

값	의미	isnull()에서 결측치로 판단하는가?
NaN, Python의 None	결측 상태를 나타내는 값	예
'None'	네 글자로 이루어진 문자열. 여기서는 코드 없음 표시	아니요
'2'	불량 유형을 나타내는 문자열 코드	아니요

빈칸을 'None'으로 바꾼다고 해서 누락된 실제 정보를 알아낸 것은 아닙니다. 정상이라 코드가 없는지, 불량인데 코드가 누락됐는지는 마지막 단계에서 Failure와 함께 확인합니다.

```
# df.FailureCode는 이 코드에서 df['FailureCode']와 같이 한 열(Series)을
# fillna('None'): 이 열의 결측치를 'None'이라는 문자열로 채운다.
# astype(str): 각 값을 문자열로 변환한다. 예: 2.0 → '2.0'
# 오른쪽의 처리 결과를 같은 열에 다시 대입하여 df에 반영한다.
df.FailureCode = df.FailureCode.fillna('None').astype(str)
```

```
# 값별 등장 횟수를 Series로 반환한다. 기본적으로 빈도가 높은 값부터 표시한다
# 불량 코드별 개수와 'None'(코드 없음)의 개수를 확인한다.
# value_counts()는 기본적으로 실제 결측치는 세지 않지만, 문자열 'None'은
df.FailureCode.value_counts()
```

	count
FailureCode	
None	839
4.0	39
3.0	35
5.0	32
1.0	31
2.0	24

dtype: int64

λ lambda와 apply() 읽기

lambda x: 결과식 은 값을 하나 받아 결과를 돌려주는 짧은 함수입니다. 아래의 apply() 는 FailureCode 의 각 값을 이 함수에 넣습니다.

x if 조건 else 다른값 은 조건이 참이면 x, 거짓이면 다른값을 결과로 선택하는 식입니다.

입력 x	조건 x == 'None'	처리	결과
'None'	True	입력 그대로 반환	'None'
'2.0'	False	float → int → str	'2'

이 분기가 있으므로 'None' 을 float() 로 바꾸려는 오류를 피할 수 있습니다. 그 외의 값은 '2.0' 처럼 숫자로 바꿀 수 있는 코드라고 가정합니다.

```
# apply(): 열의 각 값에 함수를 적용하고, 결과를 같은 인덱스의 Series로 반환
# lambda의 x에는 FailureCode의 값이 하나씩 들어간다.
# x가 'None'이면 그대로 두고, 그 외에는 소수점 없는 코드 문자열로 바꾼다.
# 변환 예: '2.0' → float로 2.0 → int로 2 → str로 '2'
# 정수로 표현할 수 있는 불량 코드라는 전제이다. int()는 반올림 함수가 아니다
df.FailureCode = df.FailureCode.apply(lambda x: x if x=='None' else
```

```
# 무작위 5개 행을 보며 FailureCode가 'None', '1', '2' 등의 형태인지 확인
df.sample(5)
```

	Lot	Temp	Viscosity	Failure	FailureCode
444	2111127987	NaN	2.4100	0	None
736	2110054079	25.7600	1.3900	0	None
624	2110125802	24.2200	1.7600	0	None
328	2110364117	75.9900	2.8500	1	None
38	2111220708	24.6000	41.6200	불량	None

```
# FailureCode의 결측치 개수와 자료형이 어떻게 달라졌는지 확인한다.
# 'None'도 문자열 값이므로 Non-Null Count에 포함된다.
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Lot              1000 non-null   int64
1   Temp             870 non-null    float64
2   Viscosity        911 non-null    float64
3   Failure          1000 non-null   object
4   FailureCode      1000 non-null   object
dtypes: float64(2), int64(1), object(2)
memory usage: 39.2+ KB
```

```
# 수치형 열의 값 범위를 다시 살펴본다.
# FailureCode는 문자열로 바뀌었으므로 기본 수치형 요약 대상에서 빠진다.
# Lot는 숫자로 저장되어도 식별자이므로 평균 자체에 공정상의 의미를 부여하지 {
df.describe()
```

	Lot	Temp	Viscosity
count	1000.0000	870.0000	911.0000
mean	2110175932.5680	31.4714	5.5346
std	677478.9190	33.0484	11.8892
min	2109021818.0000	2.4500	0.7400
25%	2109585209.0000	24.3525	1.5700
50%	2110189868.0000	25.4300	2.3900
75%	2110782208.0000	26.4400	3.2500
max	2111289797.0000	435.3500	101.8000

✓ STEP ① 완전성 지표

확인할 것: 필요한 값이 빠져 있지는 않은가?

결측치는 측정하지 못했거나 기록이 누락되는 등의 이유로 값이 없는 상태입니다. 먼저 열별 개수를 세고, 이 실습에서는 결측치가 하나라도 있는 행을 삭제합니다. 결측치 처리에는 값 채우기 등 다른 방법도 있으며, 삭제가 언제나 정답인 것은 아닙니다.

앞에서 불량 코드의 빈칸은 문자열 'None'으로 바꾸었습니다. 따라서 여기서는 결측치로 세거나 삭제하지 않습니다. **불량인데 코드가 없는 경우**는 마지막 정확성 단계에서 따로 처리합니다.

```
# isnull(): 각 칸이 결측치이면 True, 값이 있으면 False인 표를 반환한다.
# sum(): 기본적으로 각 열을 따라 더한다. True=1, False=0으로 세므로 열별
# 결과는 '열 이름 → 결측치 개수' 형태의 Series이다.
df.isnull().sum()
```

	0
Lot	0
Temp	130
Viscosity	89
Failure	0
FailureCode	0

dtype: int64

```
# 결측치가 하나라도 있는 행을 삭제한다.
# how='any': 검사한 열 중 하나라도 결측치이면 삭제
# axis=0: 열이 아니라 행을 삭제
# inplace=True: 처리 결과를 df 자체에 반영 (이 호출의 반환값은 None)
df.dropna(how='any', axis=0, inplace=True)
```

```
# 삭제 후 남은 데이터를 표시한다. 삭제된 행의 인덱스는 자동으로 다시 매겨지지
df
```

	Lot	Temp	Viscosity	Failure	FailureCode
0	2110262249	24.4400	101.8000	1	2
1	2109975633	23.9500	82.6300	1	None
2	2109024936	23.2400	1.5200	1	4
4	2109169908	23.3300	71.9200	1	None
5	2109027362	25.9600	2.3600	불량	4
...	...	...	...	...	...
951	2111214194	27.1000	41.0500	불량	None
961	2111224103	24.0500	42.4500	1	None
975	2111247872	23.5000	2.0400	정상	None
980	2111256812	44.2200	51.6800	불량	None
986	2111262910	24.1900	1.2900	1	2

791 rows × 5 columns

✓ STEP ② 유효성 지표

확인할 것: 값이 정해진 허용 범위를 만족하는가?

이 실습의 제거 조건은 온도 < 20°C 또는 온도 > 30°C 또는 점도 > 3cP입니다. 온도 20°C와 30°C, 점도 3cP는 해당 조건만으로 제거되지 않습니다. 점도의 하한 조건은 이 코드에 없습니다.

여기서 `outlier`는 이 규칙을 벗어난 행을 담은 변수입니다. 통계적 이상치 탐지 알고리즘으로 구한 결과는 아닙니다. 실제 제조 데이터에서는 극단값이 측정 오류인지, 중요한 고장 징후인지 확인한 뒤 처리 기준을 정해야 합니다.

Boolean indexing: 조건에 맞는 행 고르기

표현	결과 / 의미
<code>df.Temp < 20</code>	각 행의 온도가 20 미만인지 나타내는 True/False Series
<code>df[df.Temp < 20]</code>	조건이 True인 행만 모은 DataFrame
<code>(조건1) (조건2)</code>	둘 중 하나 이상을 만족하면 True: OR
<code>(조건1) & (조건2)</code>	두 조건을 모두 만족하면 True: AND

여러 행의 조건을 연결할 때는 `or`, `and` 대신 `|`, `&`를 사용하고 각 비교식을 괄호로 감쌉니다. 아래 코드의 바깥쪽 `df[...]`는 최종 조건이 True인 행을 선택합니다.

```
# 실습에서 정한 허용 범위를 벗어난 행을 찾는다.
# 온도 20 미만 또는 온도 30 초과 또는 점도 3 초과이면 제거 대상으로 선택한다
# 각 비교는 행마다 True/False를 만들고, |는 '또는(OR)'으로 조건을 연결한다
# df[조건]은 조건이 True인 행만 선택한다. 각 비교식은 괄호로 감싼다.
# 선택된 표를 outlier에 저장한다. 이 줄에서는 아직 df의 행을 삭제하지 않는다
outlier = df[(df.Temp < 20) | (df.Temp > 30) | (df.Viscosity > 3)]
```

```
# 제거 대상으로 선택한 행들을 확인한다.
# outlier는 조건에 맞는 행과 원래의 열을 가진 DataFrame이다.
outlier
```

	Lot	Temp	Viscosity	Failure	FailureCode
0	2110262249	24.4400	101.8000	1	2
1	2109975633	23.9500	82.6300	1	None
4	2109169908	23.3300	71.9200	1	None
6	2109032490	54.1100	2.1000	정상	None
7	2109332619	24.3900	71.1400	1	None
...	...	...	...	...	...
938	2111185019	26.3600	3.0100	정상	None
947	2111211907	23.6100	3.5500	정상	None
951	2111214194	27.1000	41.0500	불량	None
961	2111224103	24.0500	42.4500	1	None
980	2111256812	44.2200	51.6800	불량	None

296 rows × 5 columns

```
# outlier.index: 제거할 행들의 인덱스 라벨 목록
# drop(): 지정한 라벨의 행을 삭제한다. axis=0은 행, inplace=True는 df 조
# dropna()가 결측치를 기준으로 지운다면, drop()은 지정한 라벨을 기준으로 지
df.drop(outlier.index, axis=0, inplace=True)
```

✓ STEP ③ 일관성 지표

확인할 것: 같은 의미가 같은 방식으로 기록되어 있는가?

정상을 '정상' 또는 '0', 불량을 '불량' 또는 '1'로 기록하면 같은 의미가 서로 다른 값으로 집계됩니다. `map()`에 대응표를 전달하여 정수 0=정상, 1=불량으로 통일합니다.

변환 전 문자열	변환 후 정수	의미
'정상', '0'	0	정상
'불량', '1'	1	불량

변환 전후의 `value_counts()`와 `info()`를 비교하세요. `value_counts()`는 기본적으로 결측치를 제외하므로, 대응표에 없는 값이 생겼는지는 `Non-Null Count`도 함께 봐야 합니다.

```
# 정상/불량이 어떤 값으로 기록되어 있는지, 각 표현의 개수는 얼마인지 확인한다
# 예: '정상'과 '0'은 같은 의미이지만 서로 다른 값으로 집계된다.
df.Failure.value_counts()
```

	count
Failure	
0	412
정상	45
1	33
불량	5

dtype: int64

```
# df['Temp']: 이름으로 한 열을 선택하면 Series가 나온다.
# df.loc[5]: 인덱스 라벨이 5인 행을 선택한다. '여섯 번째 행'이라는 뜻은 아
# 삭제 후 라벨 5가 없으면 df.loc[5]는 오류가 나므로, 위 두 줄은 설명용 주석

# map(딕셔너리): 각 값을 {기존 값: 새 값} 대응표에 따라 바꾼다.
# 문자열 '정상', '0' → 정수 0 / 문자열 '불량', '1' → 정수 1
# 결과 Series를 원래 열에 대입한다. 따옴표가 있는 '0'과 없는 0은 자료형이 다
# 대응표에 없는 값은 결측치가 된다. 다른 데이터에 적용할 때는 실제 값과 자료형
df.Failure = df.Failure.map({'정상': 0, '0': 0,
                             '불량': 1, '1': 1})

# Failure의 자료형과 Non-Null Count를 확인한다.
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 495 entries, 2 to 986
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0    Lot             495 non-null    int64
1    Temp            495 non-null    float64
2    Viscosity       495 non-null    float64
3    Failure         495 non-null    int64
4    FailureCode     495 non-null    object
dtypes: float64(2), int64(2), object(1)
memory usage: 23.2+ KB
```

```
# 변환 후 정상(0)과 불량(1)의 개수를 확인한다.
# 정상/불량의 의미는 유지하면서 표기만 통일한 결과이다.
df.Failure.value_counts()
```

	count
Failure	
0	457
1	38

dtype: int64

✓ STEP ④ 유일성 지표

확인할 것: 같은 대상이 중복으로 기록되어 있는가?

이 실습에서는 **Lot 하나당 한 행을 남긴다**는 기준을 사용합니다. Lot가 같은 행을 중복으로 보고, 현재 행 순서에서 첫 번째 행만 남깁니다. 모든 열이 똑같은 행만 제거하는 코드가 아닙니다.

실제 데이터가 로트별 여러 시점의 측정 기록이라면 같은 Lot가 반복되는 것이 정상일 수 있습니다. 이때는 시간이나 측정 항목 등을 포함하여 중복 기준을 정해야 합니다. 또한 '첫 번째'가 가장 정확하거나 최신인 행이라는 뜻은 아닙니다.

```
# 중복 제거 전 전체 행 수를 확인한다. 제거 후의 행 수와 비교한다.
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 495 entries, 2 to 986
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Lot              495 non-null   int64
1   Temp             495 non-null   float64
2   Viscosity        495 non-null   float64
3   Failure          495 non-null   int64
4   FailureCode      495 non-null   object
dtypes: float64(2), int64(2), object(1)
memory usage: 23.2+ KB
```

```
# Lot 열에서 처음 등장한 값은 False, 같은 값이 다시 등장한 행은 True가 된다.
# sum()으로 True를 세면 첫 행을 제외한 중복 행 수가 나온다.
# 예: 같은 Lot가 3번 있으면 False, True, True → 중복 행 수는 2
# 중복된 Lot 종류의 수를 세는 것은 아니다.
df.Lot.duplicated().sum()
```

```
np.int64(40)
```

```
# subset=['Lot']: 다른 열의 값과 관계없이 Lot만 비교해 중복을 판단한다.
# keep='first': 현재 행 순서에서 처음 나온 행을 남기고 이후 중복 행은 삭제
# 새 DataFrame을 반환하므로 df에 다시 대입하여 이후 단계에서 사용한다.
df = df.drop_duplicates(subset=['Lot'], keep='first')
```

```
# 중복 제거 후 행 수가 얼마나 줄었는지 확인한다.
# 인덱스 라벨에 빈 번호가 있어도 행 수는 entries를 보고 판단한다.
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 455 entries, 2 to 975
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Lot              455 non-null   int64
```

```

1 Temp 455 non-null float64
2 Viscosity 455 non-null float64
3 Failure 455 non-null int64
4 FailureCode 455 non-null object
dtypes: float64(2), int64(2), object(1)
memory usage: 21.3+ KB

```

✓ STEP ⑤ 정확성 지표

확인할 것: 불량 여부와 불량 코드의 조합이 실습 규칙에 맞는가?

빈칸이 없더라도 두 열의 내용이 서로 맞지 않을 수 있습니다. 아래에서는 **Failure 판정을 기준으로 삼는다는** 전제로 두 경우를 처리합니다.

경우	찾는 조건	실습에서의 처리
CASE 1	정상(0)인데 불량 코드가 있음	코드를 'None'으로 수정
CASE 2	불량(1)인데 불량 코드가 없음	해당 행 삭제

CASE 1은 값을 수정하고, CASE 2는 행을 삭제한다는 차이에 주목하세요. 이것은 열 사이의 규칙을 점검하는 작업이며, 실제 제품 상태와 기록이 일치하는지까지 검증하는 것은 아닙니다. 현장에서는 원본 검사 기록 등을 확인하여 어느 값이 잘못되었는지 판단해야 합니다.

loc[행, 열] 읽기: 아래 코드는 조건에 맞는 행의 **index**를 얻은 뒤, **df.loc[행 라벨 목록, 'FailureCode']**로 수정할 칸을 지정합니다. 인덱스 라벨은 행 삭제 후에도 유지되므로 현재의 행 순번과 다를 수 있습니다.

```

# 불량 여부와 불량 코드를 나란히 보며 서로 맞지 않는 조합이 있는지 살펴본다.
# 표본 5개만으로 전체 데이터에 오류가 없다고 판단할 수는 없다.
df.sample(5)

```

	Lot	Temp	Viscosity	Failure	FailureCode
692	2110649000	26.7900	1.5400	0	None
423	2109458048	24.3500	2.4900	0	None
863	2110581504	27.0800	0.8700	0	None
320	2110580603	25.7000	2.8800	0	None
724	2111228330	25.8000	1.4100	0	None

```

# 정확성 처리 전 행 수를 기록한다. CASE 2 처리 후의 행 수와 비교한다.
df.info()

```

```

<class 'pandas.core.frame.DataFrame'>
Index: 455 entries, 2 to 975
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Lot         455 non-null   int64
1   Temp        455 non-null   float64

```

```

2 Viscosity      455 non-null    float64
3 Failure        455 non-null    int64
4 FailureCode    455 non-null    object
dtypes: float64(2), int64(2), object(1)
memory usage: 21.3+ KB

```

```

# CASE 1: 정상(0)인데 불량 코드가 기록된 행
# &: 두 조건을 모두 만족해야 선택하는 '그리고(AND)' 연산자
# !=: 같지 않다 / ==: 같다
incorrect_condi1_df = df[(df.Failure == 0) & (df.FailureCode != 'N
# loc[행 라벨, 열 이름]으로 원본 df의 해당 칸을 지정하여 값을 수정한다.
# 실습에서는 Failure 판정을 기준으로 삼아 불량 코드를 'None'으로 바꾼다.
df.loc[incorrect_condi1_df.index, 'FailureCode'] = 'None'

```

```

# CASE 2: 불량(1)인데 불량 코드가 없는 행
incorrect_condi2_df = df[(df.Failure == 1) & (df.FailureCode == 'N
# 어떤 불량 코드가 맞는지 이 데이터만으로 채울 수 없어, 실습 규칙에 따라 행을
df.drop(incorrect_condi2_df.index, axis=0, inplace = True)

```

/tmp/ipykernel_3152/1453254836.py:7: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: <https://pandas.pydata.org/pandas-docs/s>
df.drop(incorrect_condi2_df.index, axis=0, inplace = True)

```

# 최종 행 수, 열별 Non-Null Count, 자료형을 확인한다.
# CASE 1은 값 수정이므로 행 수가 유지되고, CASE 2는 행 삭제이므로 행 수가
# info()만으로 모든 품질 조건을 검증할 수는 없다. 각 단계의 조건과 처리 결과
df.info()

```

```

<class 'pandas.core.frame.DataFrame'>
Index: 449 entries, 2 to 975
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Lot          449 non-null    int64
1   Temp         449 non-null    float64
2   Viscosity    449 non-null    float64
3   Failure      449 non-null    int64
4   FailureCode  449 non-null    object
dtypes: float64(2), int64(2), object(1)
memory usage: 21.0+ KB

```

처리 흐름 돌아보기

단계	문제를 찾는 기준	이 실습의 처리
사전 정리	불량 코드의 빈칸과 표현	'None'으로 채우고 코드 문자열 통일
완전성	실제 결측치	해당 행 삭제
유효성	온도·점도의 허용 범위	범위 밖 행 삭제